Angular basics

A first application with Angular 22

What Angular provides

A TypeScript framework for web applications
Components describe sections of a page
Templates connect state to HTML
Built-in tools cover routing, forms, HTTP and testing

Modern Angular in this lesson

Standalone components with explicit imports
Signals for state and computed values
@if and @for for template control flow
Signal Forms for a first form
Vitest in new CLI projects

TypeScript essentials

Types describe the values our code expects
Interfaces name the structure of an object
The compiler checks code before execution

```
interface Course {
  id: number;
  title: string;
}

const courses: Course[] = [
  { id: 1, title: 'Web development' }
];
```

Local setup

Use a supported Node.js version and npm
For this course: Node.js 24.15.0 or newer 24.x
Install Angular CLI 22
Check the installed versions

```
node --version
npm --version
npm install -g @angular/cli@22
ng version
```

The first project

Create a client-rendered application
Open the project folder in your editor
Run the development server

```
ng new campus-app --routing --style=css --ssr=false
cd campus-app
ng serve --open
```

Project files

src/main.ts starts the application
src/app/app.ts defines the root component
app.html and app.css hold its view and styles
app.config.ts supplies application providers
app.routes.ts maps URLs to components

A complete first component

A component combines a class with a template
The selector names its HTML element

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: '<h1>{{ title }}</h1>'
})
export class App {
  title = 'Campus courses';
}
```

Template binding

Interpolation displays a value
Property binding sets an element property
Event binding calls application code

```
<h1>{{ title }}</h1>
<button [disabled]="busy">Save</button>
<button (click)="save()">Save</button>
```

Signals store changing state

Read a signal by calling it
set replaces the current value
update calculates a value from the previous one

```
import { signal } from '@angular/core';

// Inside the component class
count = signal(0);

increment() {
  this.count.update(value => value + 1);
}
```

A working counter

The template reads count()
A click calls increment()

```
<p>Selected courses: {{ count() }}</p>
<button (click)="increment()">Add one</button>
```

Computed state

computed derives a read-only value
It tracks the signals read in its calculation

```
import { computed, signal } from '@angular/core';

count = signal(0);
remaining = computed(() => 5 - this.count());

// Template
// <p>Places left: {{ remaining() }}</p>
```

Conditional content with @if

A condition controls which content appears
The template can include an alternative

```
@if (count() > 0) {
  <p>You selected {{ count() }} courses.</p>
} @else {
  <p>No courses selected.</p>
}
```

Lists with @for

Angular repeats a template for each item
track identifies each item with a stable key

```
@for (course of courses(); track course.id) {
  <p>{{ course.title }}</p>
} @empty {
  <p>No courses available.</p>
}
```

Array updates

A signal update returns a new array
Spread syntax preserves existing items

```
courses = signal<Course[]>([
  { id: 1, title: 'Web development' }
]);

addCourse(title: string) {
  const id = Date.now();
  this.courses.update(items => [
    ...items, { id, title }
  ]);
}
```

Checkpoint: predicting an update

count starts at 0
One click runs count.update(n => n + 1)
remaining is computed(() => 5 - count())

What values appear after two clicks?

Reusable child components

Generate a component for one course
Import the child in its parent
Use the child selector in the parent template

ng generate component course-item

// Parent decorator
imports: [CourseItem]

// Parent template
<app-course-item [title]="'Web development'" />

Inputs and outputs

An input receives data from a parent
An output emits an event to a parent

```
import { Component, input, output } from '@angular/core';
@Component({
  selector: 'app-course-item',
  template: `<button (click)="selected.emit()">
    {{ title() }}
  </button>`
})
export class CourseItem {
  title = input.required<string>();
  selected = output<void>();
}
```

Parent communication

The parent owns the list
Each child displays a course and emits selection

```
@for (course of courses(); track course.id) {
  <app-course-item
    [title]="course.title"
    (selected)="selectCourse(course.id)" />
}
```

Services and dependency injection

A service holds logic shared across components
Angular provides the requested instance

```
import { Service, signal } from '@angular/core';

@Service()
export class CourseStore {
  courses = signal([{ id: 1, title: 'Web development' }]);
}

// Inside a component, with inject imported
store = inject(CourseStore);
```

A first Signal Form

The signal holds the form model
form creates a tree of fields
FormField binds an input to a field

```
import { form, FormField } from '@angular/forms/signals';

// Component decorator: imports: [FormField]
// Inside the class, with signal imported
model = signal({ title: '' });
courseForm = form(this.model);

// Template
// <input [formField]="courseForm.title" />
```

Form validation

A schema defines the required title
Field state lets the template show feedback

```
import { required } from '@angular/forms/signals';

courseForm = form(this.model, path => {
  required(path.title);
});

// Template
@if (courseForm.title().touched() &&
     courseForm.title().invalid()) {
  <p>A title is required.</p>
}
```

Routing basics

Routes associate URL paths with views
The first matching route wins
RouterOutlet displays the active view

```
import { Routes } from '@angular/router';
import { CourseList } from './course-list/course-list';

export const routes: Routes = [
  { path: '', component: CourseList },
  { path: '**', redirectTo: '' }
];
```

HTTP setup and requests

Provide HttpClient at application level
A GET request returns an Observable
The request starts when code subscribes

```
import { provideHttpClient } from '@angular/common/http';
// Add provideHttpClient() to app.config.ts providers

import { HttpClient } from '@angular/common/http';
// Inside a service, with inject imported
private http = inject(HttpClient);
loadCourses() {
  return this.http.get<Course[]>('/api/courses');
}
```

Loading and error states

A useful view handles waiting and failure
Signals make these states visible to the template

```
loading.set(true);
store.loadCourses().subscribe({
  next: data => {
    courses.set(data);
    loading.set(false);
  },
  error: () => {
    errorMessage.set('Could not load courses.');
    loading.set(false);
  }
});
```

Debugging Angular

Read the first compiler error and its file location
Use browser breakpoints in component methods
Inspect requests in the Network panel
Use Angular DevTools to inspect component state
Check missing imports and signal calls first

Testing the behavior

New CLI projects use Vitest
Test user-visible behavior and important state changes

```
import { describe, expect, it } from 'vitest';

describe('counter', () => {
  it('increments the selected count', () => {
    const app = new App();
    app.increment();
    expect(app.count()).toBe(1);
  });
});

// Terminal: ng test
```

Guided exercise: course list

15 minutes in pairs
Display a list of course titles
Add a course from a labeled input
Reject blank titles
Show the total with computed
Show an empty message when the list has no items

Exercise starter: component class

Replace src/app/app.ts with this starter

```
import { Component, computed, signal } from '@angular/core';
@Component({
  selector: 'app-root',
  templateUrl: './app.html'
})
export class App {
  courses = signal<{ id: number; title: string }[]>([]);
  total = computed(() => this.courses().length);
  private nextId = 1;
  addCourse(raw: string) {
    // TODO: trim, reject blank, then update the list
  }
}
```

Exercise starter: template

Replace src/app/app.html
The #title reference points to the input element

```
<h1>Campus courses ({{ total() }})</h1>
<label for="title">Course title</label>
<input id="title" #title />
<button (click)="addCourse(title.value); title.value = ''">
  Add course
</button>
@for (course of courses(); track course.id) {
  <p>{{ course.title }}</p>
} @empty {
  <p>No courses yet.</p>
}
```

Common beginner mistakes

count instead of count() when reading a signal
Array mutation without a signal set or update
A child, directive or pipe missing from imports
A route without a RouterOutlet
An HTTP request without a subscription
Copying examples for a different Angular version

Exit questions

What belongs in the component class?
What does [disabled] bind?
Why does @for need track?
How does a child notify its parent?
When should shared logic move into a service?

Next steps and official resources

Interactive tutorial: angular.dev/tutorials/learn-angular
Components: angular.dev/guide/components
Signals: angular.dev/guide/signals
Forms: angular.dev/guide/forms/signals/overview
Routing and HTTP: angular.dev/guide/routing and /guide/http
Compatibility: angular.dev/reference/versions